Contents

1 Introduction and Objectives **1**
- 1.1 Introduction . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Project Objectives . . . 1

2 Project Category, Scope, and Feasibility **2**
- 2.1 Project Category . . . 2
- 2.2 Scope of the Proposed Solution . . . 2
  - 2.2.1 In-Scope Capabilities (Proposed System Deliverables) . . . 2
  - 2.2.2 Out-of-Scope Capabilities (Future Phase Boundaries) . . . 2

3 Functional and Non-Functional Requirements **3**
- 3.1 Proposed Functional Requirements . . . 3
- 3.2 Proposed Non-Functional Requirements . . . 3

4 Technical Specifications and Architecture **4**
- 4.1 Proposed Technology Stack . . . 4
- 4.2 Hardware and Software Requirements . . . 4
- 4.3 System Architecture and Data Flow Models . . . 4
- 4.4 Proposed API Interface Specification . . . 7

5 Analysis and Data Models **8**
- 5.1 Entity-Relationship Model . . . 8
- 5.2 User Workflow and Activity Diagram . . . 8
- 5.3 UML State Machine Model . . . 10

6 Database Design and Schema Specification **11**
- 6.1 Database Design Principles . . . 11
- 6.2 Core User and Daily Activity Schema . . . 11
- 6.3 Category Lookup and Mapping Tables . . . 11
- 6.4 Workout and Diet Schedule Catalogues . . . 12

7 Modular Structure and Implementation Methodology **13**
- 7.1 Module Decomposition and Planned Effort . . . 13
- 7.2 Process Logic and Core Algorithms . . . 13
- 7.3 Core Data Structures . . . 14

8 Project Planning, Scheduling, and Risk Management **15**
- 8.1 Methodology and Phased Execution . . . 15
- 8.2 Proposed Gantt Schedule . . . 15
- 8.3 PERT Precedence Network and Critical Path . . . 15

9 Network Architecture and Security Design **16**
- 9.1 Proposed Network Deployment Topology . . . 16
- 9.2 Multi-Layered Security Architecture . . . 16

10 Future Scope, Expected Outcomes, and Conclusion **17**
- 10.1 Future Scope . . . 17
- 10.2 Expected Project Outcomes and Deliverables . . . 17

10.3 Conclusion 17

List of Figures

4.1 Level-0 Context Data Flow Diagram (DFD). 4
4.2 Level-1 System Data Flow Diagram. 5
4.3 Level-2 DFD: Decomposition of Authentication and Profile Management (Process 1.0). 5
4.4 Level-2 DFD: Decomposition of Plan Retrieval (Process 2.0). 6
4.5 Level-2 DFD: Decomposition of Daily Activity Logging and History Review (Process 3.0). 6

5.1 Proposed Entity-Relationship (ER) Diagram. 8
5.2 UML Activity Diagram of the User Workflow. 9
5.3 UML State Machine for Daily Activity Submission. 10

8.1 Proposed Eight-Week Project Gantt Schedule. 15
8.2 Proposed Project PERT Precedence Network. 15

9.1 Proposed Network Deployment Topology and Trust Boundaries. 16

List of Tables

3.1 Proposed Functional Requirements Specification 3

4.1 Proposed Technology Stack Specifications 4
4.2 Proposed RESTful HTTP API Route Inventory 7

6.1 Core User and Daily Activity Relational Schema 11
6.2 Lookup and Recommendation Mapping Tables 11

7.1 Proposed System Modules and Development Allocation 13
7.2 Principal Data Structures Across Application Layers 14

Chapter 1

Introduction and Objectives

1.1 Introduction

People who exercise independently may use separate tools to keep profile details, routines, hydration notes, and activity records. A focused web application can make these records easier to review together. This proposal addresses that information-management need for students and other users who want a simple personal activity log.

The **Formwell** project proposes a browser application that brings profile baselines, curated routines, meal-plan content, and daily activity logs into one interface. The planned architecture separates the browser frontend, Go HTTP API, and PostgreSQL persistence layer. Performance and reliability will be evaluated against defined workloads during development.

The planned capabilities include account registration and sign-in, a profile baseline with BMI, rule-based selection of cardio, strength, and meal-plan content, daily water and step entry, exercise progress logging, and review of activity by selected date. The design specifies adaptive password hashing, signed JWT bearer authentication, token-derived authorization, strict server-side validation, and minimal profile responses.

1.2 Problem Statement

A person who records activity across separate tools may find it difficult to connect daily measurements with suggested routines or review earlier entries. Formwell will provide one account-based workflow for profile details, predefined plans, daily measurements, and selected-date summaries. The system will validate inputs, enforce user ownership, and preserve one daily record per user and date.

1.3 Project Objectives

The specific, measurable technical objectives proposed for the Formwell project are:

1. **Responsive User Interface:** Design and build an accessible, responsive web interface using HTML5, modern CSS3, and vanilla JavaScript that allows frictionless user registration, authentication, dashboard inspection, and metric logging.
2. **Go HTTP API:** Design and implement a lightweight JSON API in Go using the standard HTTP library, explicit request handling, and a narrowly configured CORS policy.
3. **Biometric Baseline Classification Engine:** Implement a rule-based assessment algorithm that computes Body Mass Index (BMI) from user height and weight, assigns standardized physical categories, and maps users to pre-curated cardio and strength regimens.
4. **Daily Activity Tracking:** Develop a persistence workflow for water intake, step counts, and completion counts for thirteen defined exercise fields, enforcing one record per user and calendar date.
5. **Relational Database Design and Normalization:** Construct a normalized relational schema in PostgreSQL using GORM ORM, incorporating primary keys, foreign key constraints with cascade rules, and composite indexes.
6. **Security Framework:** Implement adaptive bcrypt password hashing, HMAC-SHA-256 signed JSON Web Tokens (JWT), token-derived subject authorization, and server-side validation.
7. **Historical Progress Analytics:** Deliver an interactive date-driven reporting module that retrieves historical daily activity records and visualizes progress against recommended daily targets.

Chapter 2

Project Category, Scope, and Feasibility

2.1 Project Category

The proposed project falls within the following computer science and software engineering domains:

- **RDBMS-Backed Information Systems:** The core data persistence layer relies on PostgreSQL, leveraging relational modeling, normalization, composite indexes, and foreign key cascades to ensure absolute data consistency.
- **Full-Stack Web Architecture:** The project implements a distributed client-server architecture separating presentation (HTML5/CSS3/JavaScript) from business logic (Go HTTP REST API) and data storage.
- **Structured and Modular Software Engineering:** The Go backend utilizes modular packages, explicit data structures (structs), and interface-driven database abstractions to promote high cohesion and low coupling.
- **Network Protocols and Stateless APIs:** Client-server communication occurs strictly over HTTP using standard JSON payloads, stateless bearer tokens, and standardized status codes.

2.2 Scope of the Proposed Solution

2.2.1 In-Scope Capabilities (Proposed System Deliverables)

The planned capabilities for the project lifecycle are listed below. For planning, the first release assumes a modern JavaScript-enabled browser, one API service, and PostgreSQL; a single calendar-date and timezone policy will be defined before activity logging is implemented.

1. **Authentication & Profile Management:** User registration with physiological baselines (age, gender, height, weight), automated BMI derivation, secure login with JWT issuance, and session termination.
2. **Curated Workout & Diet Regimens:** Rule-based catalog mapping users into age and BMI tiers, displaying pre-seeded cardio schedules, strength routines, and structured meal recommendations.
3. **Daily Metrics Ingestion:** Interactive logging forms for water intake, steps, and completion counts for the thirteen defined exercise fields.
4. **Historical Progress Review:** Date-selected reporting views allowing users to inspect past performance against pre-established targets.
5. **Containerized Environment:** Docker and Docker Compose configuration for reproducible local development; production hosting will be specified separately if included in the approved scope.

2.2.2 Out-of-Scope Capabilities (Future Phase Boundaries)

To ensure project delivery within academic timelines, the following elements are designated as out-of-scope for the initial release:

- Direct hardware integration with wearable Internet-of-Things (IoT) sensors (e.g., Apple HealthKit, Fitbit BLE APIs).
- Automated medical diagnostic claims or personalized clinical decision-support algorithms.
- Native mobile applications compiled for iOS (Swift) or Android (Kotlin).
- Commercial payment processing, subscription tiering, or multi-tenant enterprise administration.

Chapter 3

Functional and Non-Functional Requirements

3.1 Proposed Functional Requirements

Table 3.1 defines the proposed software capabilities using testable shall-statements.

Table 3.1: Proposed Functional Requirements Specification

Req. ID	Requirement Specification
FR-01	The system shall allow new users to register by submitting a unique username, password, personal name, gender, age, height, and weight.
FR-02	The system shall calculate the user's Body Mass Index (BMI) upon registration and store the baseline profile within the database.
FR-03	The system shall authenticate user credentials against bcrypt-hashed passwords and issue an HMAC-SHA-256 signed JWT bearer token upon successful verification.
FR-04	The system shall validate the bearer token on all protected API endpoints and reject unauthenticated or expired requests with HTTP 401 Unauthorized.
FR-05	The system shall retrieve and display the authenticated user's profile baseline upon request, strictly omitting sensitive credential fields from the response.
FR-06	The system shall map user biometric parameters to appropriate pre-seeded cardio schedules, strength routines, and nutritional diet plans.
FR-07	The system shall provide an interactive daily input interface allowing users to record daily water consumption and walking steps.
FR-08	The system shall allow users to record completion counts for squats, deadlifts, pushups, pullups, jumping jacks, weightlifting, bench press, lunges, leg press, running, cycling, swimming, and walking.
FR-09	The system shall persist daily logs under a composite unique constraint of username and date, updating existing records if re-submitted on the same date.
FR-10	The system shall retrieve historical activity records for any user-selected date and render progress bars comparing logged activity against recommended targets.
FR-11	The system shall enforce subject authorization, ensuring users can only read, write, or modify records that correspond to their verified token identity.
FR-12	The system shall provide a sign-out workflow that clears the browser's active session token and cached user-specific state. Token revocation, if required, shall be addressed in the session design.

3.2 Proposed Non-Functional Requirements

- **Security:** Passwords shall be hashed using bcrypt (work factor ≥ 10); protected API endpoints shall require JWT authorization; request inputs shall be validated server-side; sensitive attributes shall be omitted from logs and public API responses.
- **Data Integrity:** Database foreign keys must enforce referential integrity; user/date uniqueness must be guaranteed; numerical values (water, steps, reps) must be constrained to non-negative bounds.
- **Performance & Latency:** Define representative read and write workloads, measure API response times in the target environment, and report the results as part of project evaluation.
- **Usability & Responsiveness:** The web interface shall remain usable across agreed mobile and desktop viewport sizes; the project shall review navigation, form feedback, and layout behavior at representative widths, including 360px and 1920px.
- **Maintainability & Modularity:** Backend logic must strictly decouple HTTP handling, business routing, and database persistence; Go modules and structs must adhere to clean architectural conventions.
- **Portability:** The development environment shall document supported Go and PostgreSQL versions and provide reproducible local database setup through Docker Compose. Cross-platform builds will be treated as an evaluation item if required by the deployment environment.

Chapter 4

Technical Specifications and Architecture

4.1 Proposed Technology Stack

Table 4.1 lists the proposed technology stack and the role of each component.

Table 4.1: Proposed Technology Stack Specifications

Layer / Component	Proposed Technology	Rationale and Technical Role
Frontend Presentation	HTML5, CSS3, Modern JavaScript (ES6+)	Lightweight, zero-dependency, highly responsive client interface executable in any modern web browser.
Application Server	Go (Golang) v1.22+	High-concurrency compiled backend, strong type safety, low memory footprint, and native HTTP performance.
Routing & Middleware	Go Standard Library (`net/http`)	Standardized ServeMux routing, custom CORS handlers, and JWT authentication middleware.
Authentication	JWT (`golang-jwt/jwt/v5`) & bcrypt	Secure token generation with HS256 algorithm and adaptive cryptographic credential hashing.
Persistence / ORM	PostgreSQL 16 with GORM v1.25+	Enterprise relational storage, ACID compliance, structured object-relational mapping, and automated schema migration.
Containerization	Docker & Docker Compose	Reproducible local database setup; the API image may be used for container-based deployment after environment configuration is defined.

4.2 Hardware and Software Requirements

- **Development System Baseline:** 64-bit multi-core processor (x86-64 or Apple Silicon ARM64), 8 GB RAM, 10 GB available SSD storage, macOS 14+, Ubuntu 22.04 LTS, or Windows 11 with WSL2.
- **Software Development Environment:** Go SDK 1.22 or higher, Docker Engine 24+ and Docker Compose v2, Git version control, Visual Studio Code , and standard web browsers.
- **Target Deployment Environment:** Linux container host (Ubuntu Server / Alpine Linux container), Docker runtime, persistent PostgreSQL volume.

4.3 System Architecture and Data Flow Models

The proposed system will use a three-tier client-server architecture comprising the browser presentation layer, Go application layer, and PostgreSQL persistence layer. The DFDs use standard data-flow notation: rectangular external entities, numbered rounded-rectangle processes, labeled directed data flows, and D1–D3 data stores.

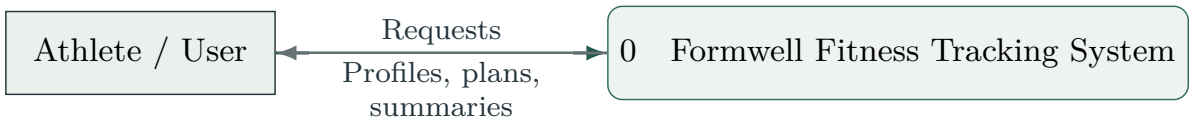


Figure 4.1: Level-0 Context Data Flow Diagram (DFD).

The Level-1 DFD (Figure 4.2) decomposes process 0 into authentication and profile management (1.0), plan

retrieval (2.0), and daily activity logging and history review (3.0). The user symbol is repeated for layout clarity; each copy represents the same external entity. Every data store exchanges data through a process, with no direct store-to-store flows.

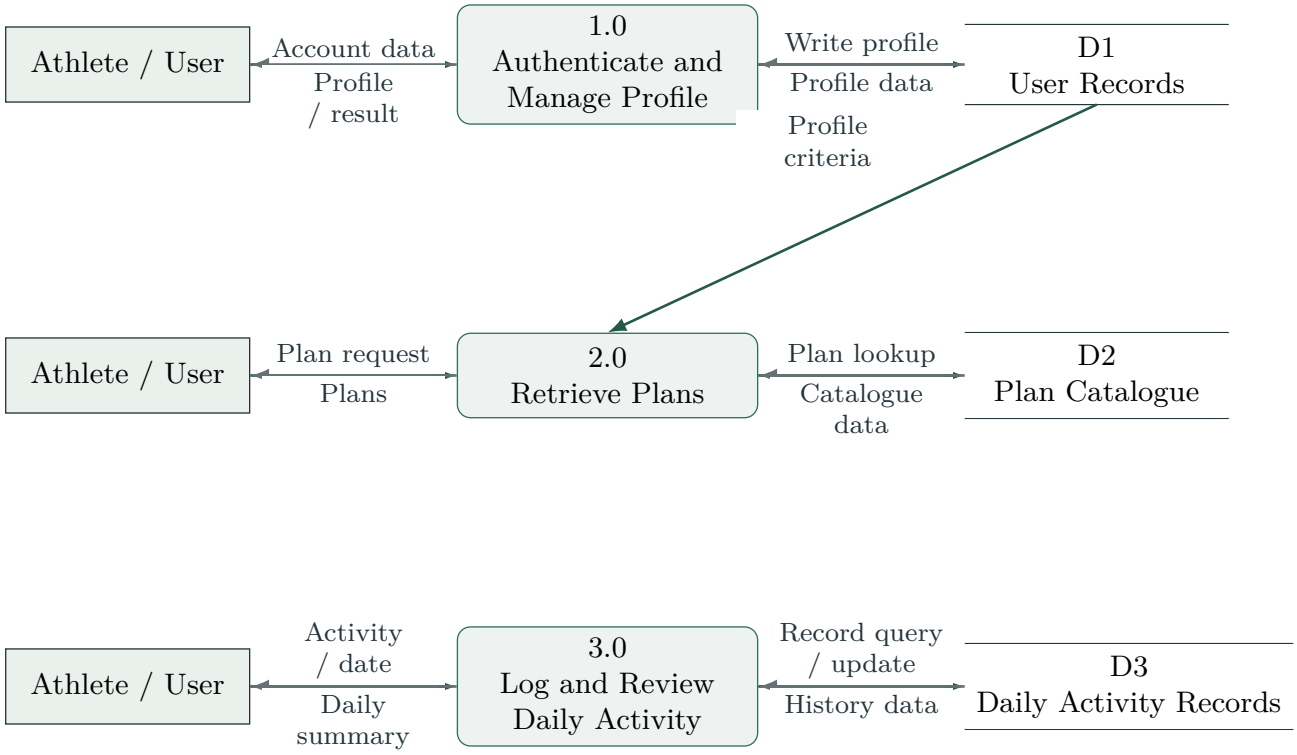


Figure 4.2: Level-1 System Data Flow Diagram.

The following Level-2 DFDs decompose each Level-1 process. Each process exchanges data only with an external entity or a data store through a process; no external-entity-to-store or store-to-store flow is shown.

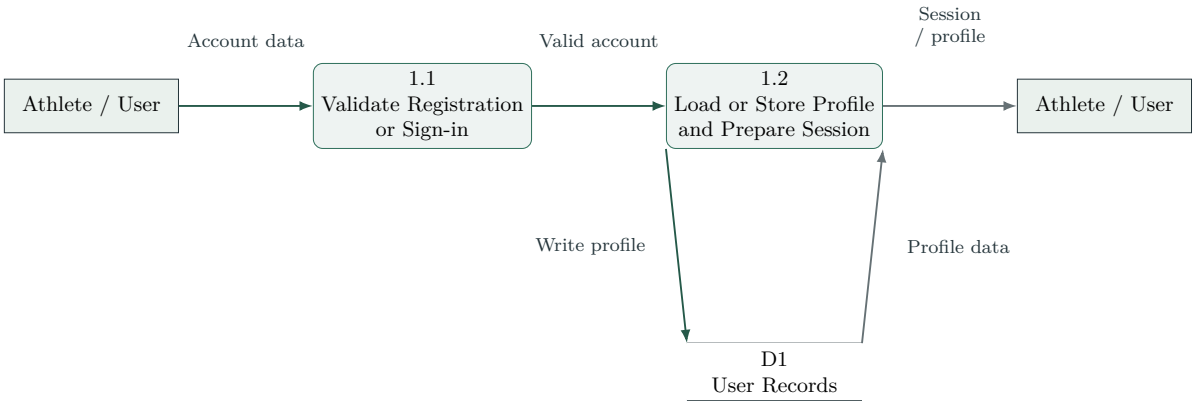


Figure 4.3: Level-2 DFD: Decomposition of Authentication and Profile Management (Process 1.0).

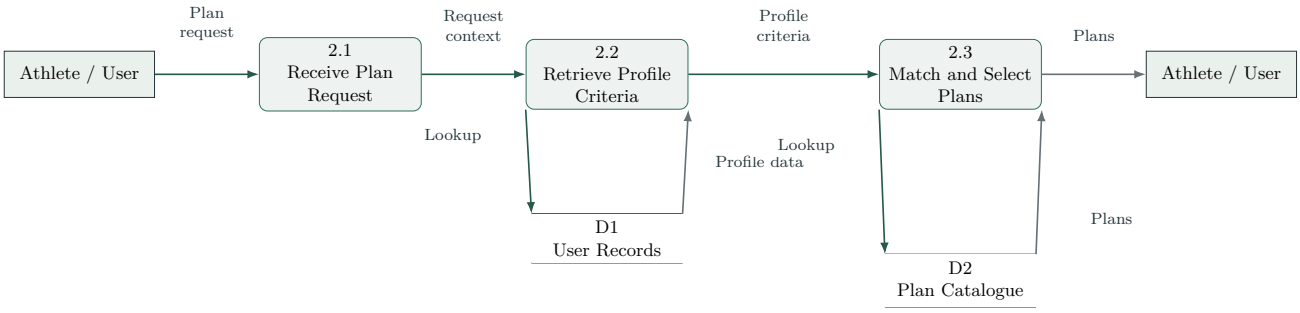


Figure 4.4: Level-2 DFD: Decomposition of Plan Retrieval (Process 2.0).

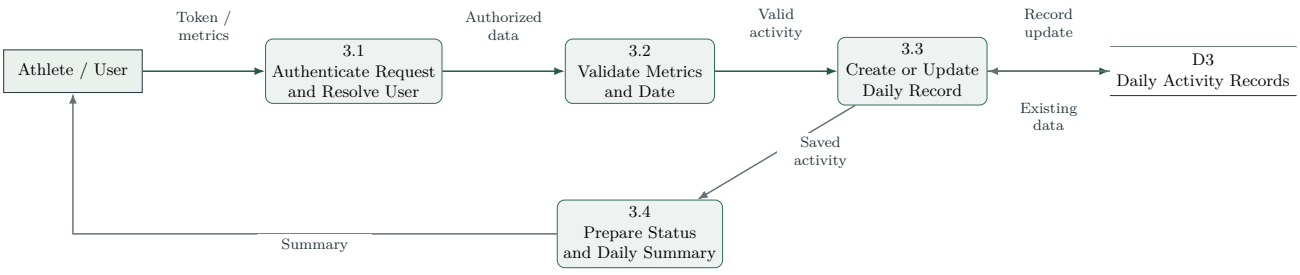


Figure 4.5: Level-2 DFD: Decomposition of Daily Activity Logging and History Review (Process 3.0).

4.4 Proposed API Interface Specification

The proposed browser-to-API interface uses JSON over HTTP during local development and HTTPS in deployment. Table 4.2 specifies resource-oriented routes and methods for the planned implementation.

Table 4.2: Proposed RESTful HTTP API Route Inventory

Endpoint Path	**Method**	**Protection**	**Functional Purpose**
`/signup`	POST	Public	Register user profile and store hashed credentials.
`/login`	POST	Public	Verify credentials and issue signed JWT token.
`/check-auth`	GET	Bearer JWT	Validate active session token status.
`/users/me`	GET	Bearer JWT	Retrieve a sanitized profile baseline for the authenticated subject.
`/users/me/workout-plan`	GET	Bearer JWT	Retrieve the recommended strength training schedule.
`/users/me/cardio-plan`	GET	Bearer JWT	Retrieve the recommended cardio exercise schedule.
`/users/me/diet-plan`	GET	Bearer JWT	Retrieve the recommended weekday meal-plan content.
`/users/me/activity`	PUT	Bearer JWT	Create or update daily water, steps, and exercise counts.
`/users/me/activity/today`	GET	Bearer JWT	Retrieve current-day logged metrics and measurements.
`/users/me/activity/\{date\}`	GET	Bearer JWT	Retrieve historical metrics for an explicit calendar date.
`/users/me/targets/today`	GET	Bearer JWT	Retrieve current-day targets and completion values.
`/users/me/targets/\{date\}`	GET	Bearer JWT	Retrieve targets and completion values for the selected date.

Chapter 5

Analysis and Data Models

5.1 Entity-Relationship Model

The logical data model of Formwell is depicted in Figure 5.1. Solid connectors show database foreign-key relationships. Each workout mapping references exactly one body type, age group, cardio tier, and strength tier; each lookup row may be referenced by zero or more mappings. Daily logs belong to one user, and the unique constraint on (username, date) permits at most one log per user per date. Exercise targets reference an age group through a unique foreign key, so an age group has at most one target row. Schedule and diet tables are seeded catalogs selected by application code; they have no declared foreign keys to the tier tables and are therefore shown separately.

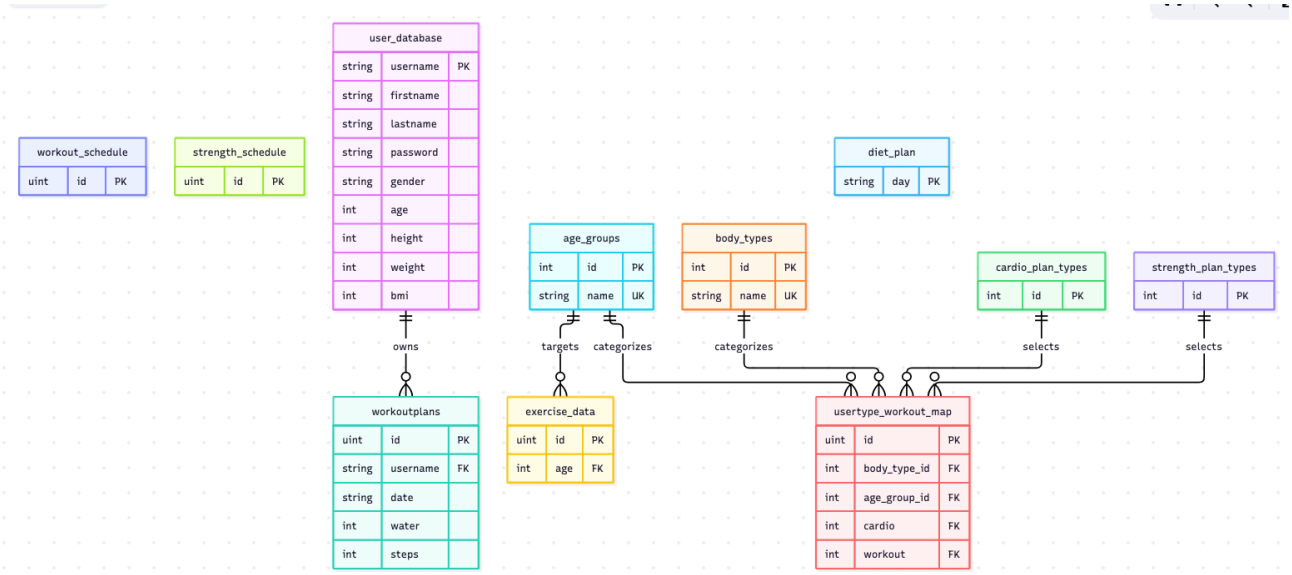


Figure 5.1: Proposed Entity-Relationship (ER) Diagram.

5.2 User Workflow and Activity Diagram

Figure 5.2 uses UML activity notation. It shows session validation, the authenticated dashboard, guarded navigation choices, and the activity paths available to the user.

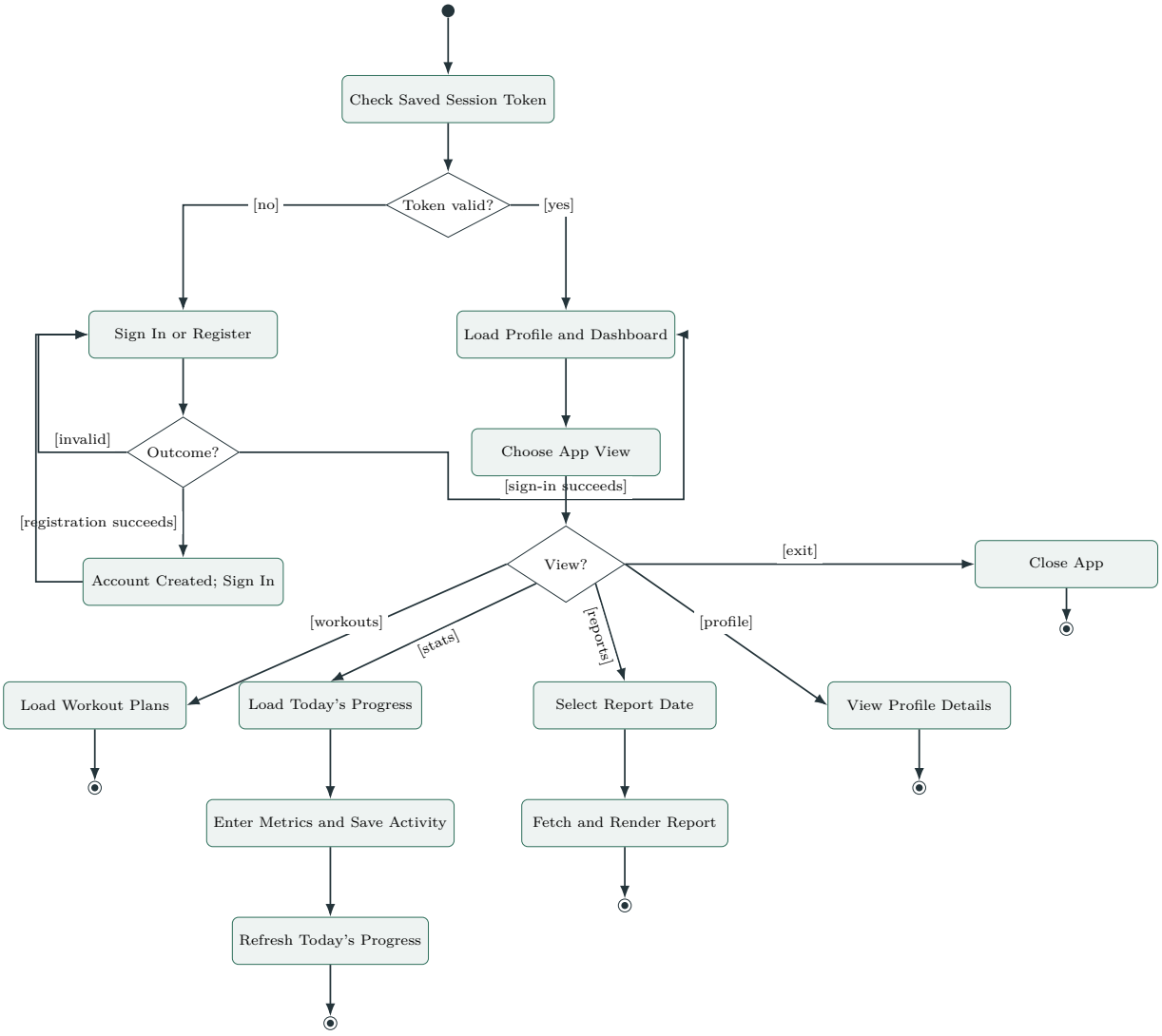


Figure 5.2: UML Activity Diagram of the User Workflow.

5.3 UML State Machine Model

Figure 5.3 uses UML state-machine notation to show the lifecycle of a daily activity submission. Guarded transitions distinguish malformed input, a missing or existing daily record, and successful or failed writes.

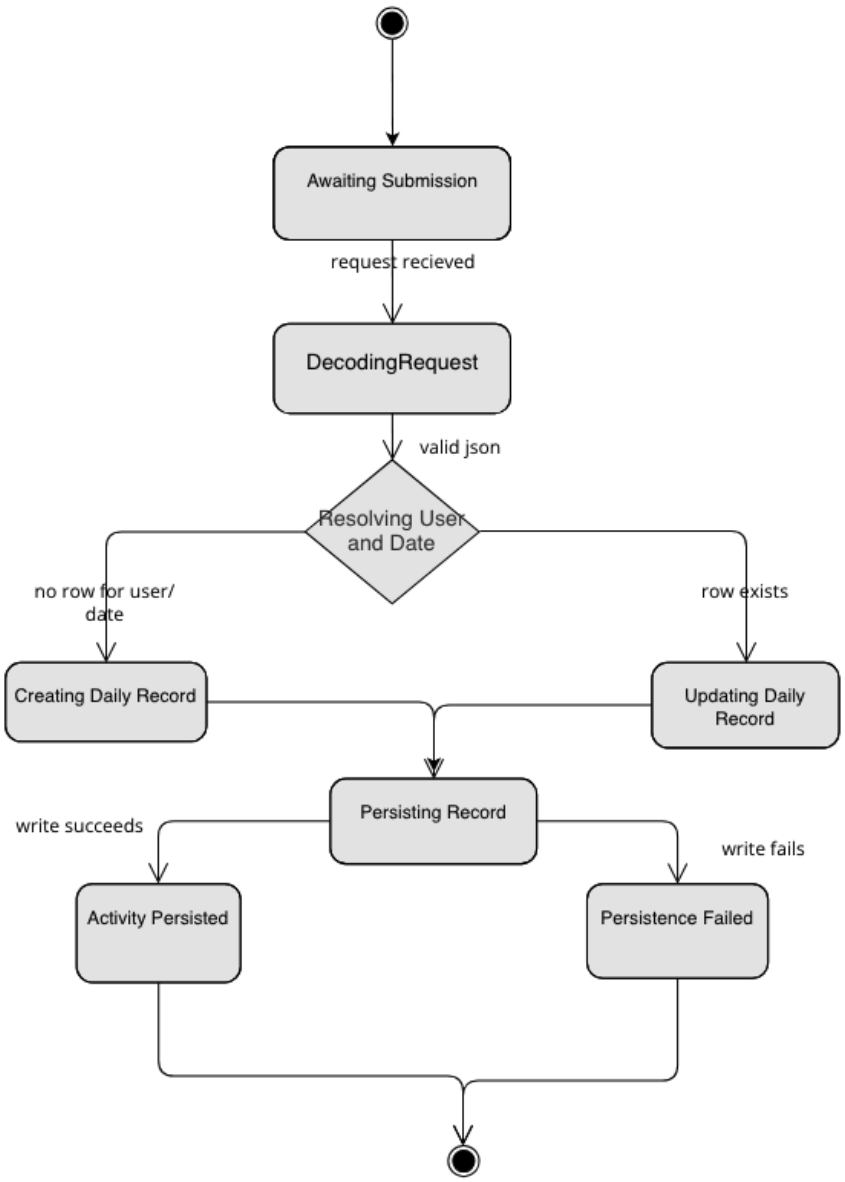


Figure 5.3: UML State Machine for Daily Activity Submission.

Chapter 6

Database Design and Schema Specification

6.1 Database Design Principles

The proposed database separates user profiles, category lookups, category mappings, schedule catalogues, and user-day activity records. Primary and foreign keys and a composite unique key on user and date will protect core relationships. The schedule catalogues retain separate category-specific tables to reflect the project design; their duplication will be reviewed during schema refinement. The design will use suitable keys and constraints to protect data integrity. User and activity records will include creation and update timestamps; security events may be recorded without credentials or bearer tokens, subject to a retention policy.

6.2 Core User and Daily Activity Schema

Table 6.1 specifies the core relational schema for user authentication profiles and daily workout tracking.

Table 6.1: Core User and Daily Activity Relational Schema

Table Name / Key	Columns, Data Types, and Constraints
`user_database` **PK:** `username`	`username`: VARCHAR(64) PRIMARY KEY, NOT NULL; `firstname`, `lastname`: VARCHAR(64), NOT NULL; `password`: VARCHAR(255), NOT NULL (stores adaptive bcrypt hash); `gender`: VARCHAR(16), NOT NULL; `age`: INT, NOT NULL, CHECK (age $>$ 0); `height`: INT, NOT NULL, CHECK (height $>$ 0); `weight`: INT, NOT NULL, CHECK (weight $>$ 0); `bmi`: DECIMAL(5,2), NOT NULL, derived from height and weight; category boundaries shall be applied to the unrounded value.
`workoutplans` **PK:** `id` **FK:** `username`	`id`: BIGSERIAL PRIMARY KEY; `username`: VARCHAR(64) NOT NULL, FK references `user_database(username)` ON UPDATE CASCADE ON DELETE CASCADE; `date`: DATE NOT NULL; **Composite Unique Index:** `idx_user_date` on (`username`, `date`).
`workoutplans` (Daily Metrics & Exercise Counts)	Integer Daily Metrics: `water` (ml, non-negative), `steps` (count, non-negative). Exercise Repetition Counts: `squatsdone`, `deadliftdone`, `pushupsdone`, `pullupsdone`, `jumpingjacksdone`, `weightliftdone`, `benchpressdone`, `lungesdone`, `legpressdone`, `runningdone`, `cyclingdone`, `swimmingdone`, `walkingdone` (all INT, default 0, CHECK $\geq$ 0). Each record shall also include `created_at` and `updated_at` timestamps.
`workoutplans` (Target Columns)	Target Baseline Columns: `squats`, `deadlift`, `pushups`, `pullups`, `jumpingjacks`, `weightlift`, `benchpress`, `lunges`, `legpress`, `running`, `cycling`, `swimming`, `walking` (VARCHAR(32), recommended baseline goals).

6.3 Category Lookup and Mapping Tables

To support deterministic recommendation, the system employs lookup and mapping tables as detailed in Table 6.2.

Table 6.2: Lookup and Recommendation Mapping Tables

Table Name	Columns, Data Types, and Constraints
`body_types`	`id`: INT PRIMARY KEY; `name`: VARCHAR(32) NOT NULL, UNIQUE. Seeded as rule-based BMI bands: `type_1` below 18.5, `type_2` from 18.5 to below 25, and `type_3` at or above 25; these are selectors for catalogue content, not clinical assessments.
`age_groups`	`id`: INT PRIMARY KEY; `name`: VARCHAR(32) NOT NULL, UNIQUE. Seeded with age tiers (`group_1`: below 20; `group_2`: 20–40 inclusive; `group_3`: above 40).

Table Name	Columns, Data Types, and Constraints
cardio_plan_types	id: INT PRIMARY KEY. Seeded identifiers 1–3 corresponding to cardio schedule tiers.
strength_plan_types	id: INT PRIMARY KEY. Seeded identifiers 1–3 corresponding to strength schedule tiers.
usertype_workout_map	id: BIGSERIAL PRIMARY KEY; body_type_id, age_group_id, cardio, workout: INT NOT NULL; **Composite Unique Index:** on (body_type_id, age_group_id); FKs referencing lookup tables with ON UPDATE CASCADE ON DELETE RESTRICT.
exercise_data	id: BIGSERIAL PRIMARY KEY; age: INT NOT NULL UNIQUE, FK to age_groups(id); thirteen target fields defining baseline daily goals for each age bracket.

6.4 Workout and Diet Schedule Catalogues

The proposed database will store the schedule catalogues in category-specific relations, with a schema review to consolidate shared fields where practical:

- **Cardio Schedules (workout_schedule1-3):** Contain structured routines for each day of the week (monday through sunday, VARCHAR(255) NOT NULL).
- **Strength Schedules (strength_schedule1-3):** Detail specific muscle group splits and exercises for all seven calendar days.
- **Dietary Plans (diet_plan1-3):** Map days of the week (day: PK) to balanced nutritional recommendations covering breakfast, lunch, and dinner.

Chapter 7

Modular Structure and Implementation Methodology

7.1 Module Decomposition and Planned Effort

The proposed system is decomposed into nine functional modules (M1–M9). The illustrative effort estimate is 36 person-days across the planned eight-week lifecycle, as shown in Table 7.1; it will be reviewed with the project guide.

Table 7.1: Proposed System Modules and Development Allocation

ID	Module Name	Core Functional Responsibility	Effort	Phase
M1	Identity & Session	User signup, bcrypt password hashing, login verification, JWT token issuance, and logout session clearing.	4 days	Phase 2
M2	Profile & Baseline	Height/weight ingestion, automated BMI calculation, physical baseline storage, and profile dashboard rendering.	3 days	Phase 2
M3	Plan Catalog Engine	Biometric classification, age/BMI mapping algorithm, and retrieval of curated cardio, strength, and diet schedules.	5 days	Phase 2
M4	Daily Activity Ingestion	Forms for water, steps, and 13 exercise counts; input sanitization; atomic upsert under user/date unique key.	4 days	Phase 3
M5	Progress Analytics	Date-picker query engine, historical record retrieval, progress calculation against target baselines, and report rendering.	3 days	Phase 3
M6	Core REST API	Go ServeMux routing, CORS policy middleware, JSON request decoding, and GORM database repository interface.	5 days	Phase 2
M7	Responsive UI	Modular HTML5 templates, fluid CSS3 Grid/Flexbox styling, client-side validation, and asynchronous fetch handlers.	5 days	Phase 3
M8	Security & Testing	Subject-to-token authorization checks, unit tests, integration testing, and vulnerability remediation.	5 days	Phase 4
M9	DevOps & Packaging	Dockerfile multi-stage builds, Docker Compose configuration, automated database seeding, and final documentation.	2 days	Phase 5
		Total Estimated Development Effort:	**36 days**	

7.2 Process Logic and Core Algorithms

The proposed system will use three deterministic processes:

1. **BMI Derivation and Rule-Based Category Assignment:**

$$\text{BMI} = \frac{\text{weight (kg)}}{(\text{height (m)})^2} = \frac{\text{weight (kg)} \times 10{,}000}{(\text{height (cm)})^2} \tag{7.1}$$

   For the project catalogue, the derived BMI will be assigned to these rule-based tiers (not medical diagnoses): lower BMI band (BMI < 18.5, `type_1`), middle BMI band ($18.5 \leq$ BMI < 25.0, `type_2`), and upper BMI band (BMI ≥ 25.0, `type_3`).
2. **Deterministic Routine Mapping Algorithm:** Given the authenticated user's body type $B \in \{1, 2, 3\}$ and age group $A \in \{1, 2, 3\}$, the system queries the associative relation:

$$(B, A) \xrightarrow{\text{usertype_workout_map}} (\text{CardioTier}, \text{StrengthTier}) \tag{7.2}$$

   The resolved tier identifiers determine the exact schedule and diet catalogs delivered to the client.
3. **Idempotent Daily Logging Algorithm:** When a daily log payload is received for user U and calendar date D, the system queries `workoutplans` for the tuple (U, D). The API will use the composite unique key and a transactional upsert to prevent duplicate records. The record will include or reference the applicable target values from `exercise_data`.

7.3 Core Data Structures

Table 7.2 outlines the primary data structures across the application layers.

Table 7.2: Principal Data Structures Across Application Layers

System Layer	Data Structure	Purpose and Application
Browser Client	Plain JavaScript Objects	Dynamic form state, cached profile attributes, and asynchronous JSON payloads.
Browser State	In-memory session state	Short-lived access token and active profile data; persistent browser storage of bearer tokens will be avoided.
API Layer	Go Structs (`User`, `Workoutplan`)	Strongly typed domain models with JSON and GORM struct tags.
API Context	JWT Claims Struct	Contextual user claims (`Username`, `ExpiresAt`) carried within authenticated request contexts.
Persistence Layer	PostgreSQL Relations	ACID-compliant relational tables enforcing constraints and indexing.

Chapter 8

Project Planning, Scheduling, and Risk Management

8.1 Methodology and Phased Execution

The project will be executed using an iterative, phased software development lifecycle organized into five distinct phases over eight calendar weeks:

- **Phase 1: Inception & Architectural Specification (Weeks 1–2):** Requirements finalization, database normalization, API contract definition, and project approval.
- **Phase 2: Core Backend & Persistence Development (Weeks 2–4):** Go HTTP API scaffolding, PostgreSQL GORM integration, bcrypt authentication, and plan engine.
- **Phase 3: Frontend Implementation & Integration (Weeks 3–6):** Responsive UI views, asynchronous client request helpers, daily logging forms, and dashboard analytics.
- **Phase 4: Security Hardening & Quality Assurance (Weeks 6–7):** Subject authorization enforcement, comprehensive unit and API integration testing, and bug remediation.
- **Phase 5: Packaging, Deployment & Evaluation (Weeks 7–8):** Docker containerization, final user acceptance testing, and academic project report preparation.

8.2 Proposed Gantt Schedule

Figure 8.1 illustrates the planned timeline and task overlaps across the eight-week execution window.

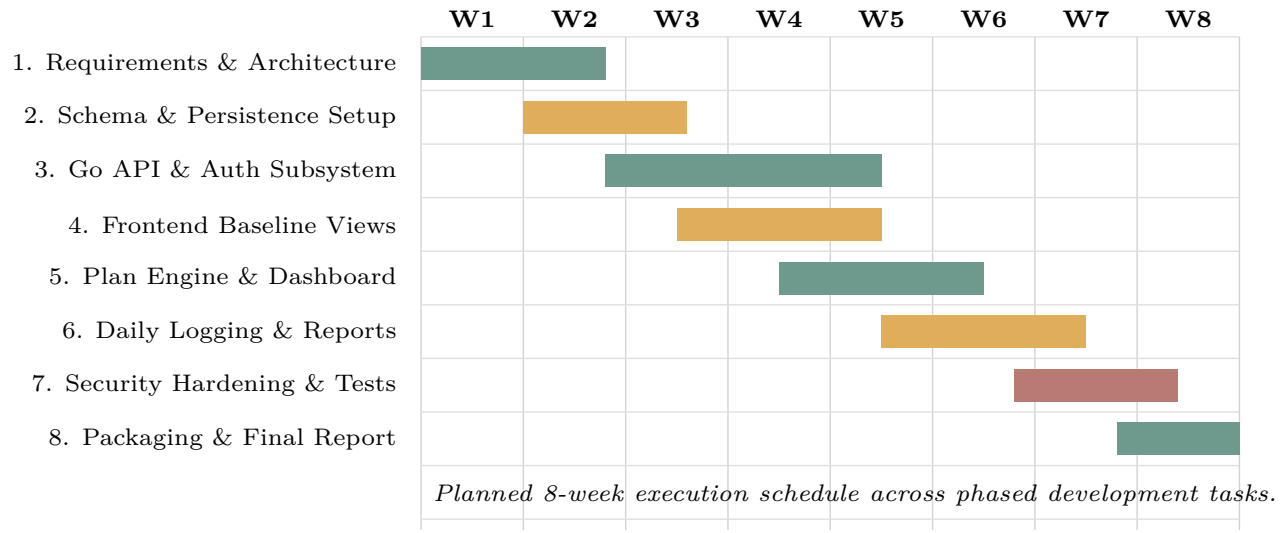


Figure 8.1: Proposed Eight-Week Project Gantt Schedule.

8.3 PERT Precedence Network and Critical Path

Figure 8.2 models the task dependencies and precedence constraints. The critical path spans activities A → B → C → D → E → F → G → H, totaling 38 sequential working days. At five working days per week, this is approximately eight work weeks; the Gantt schedule uses limited task overlap and remains subject to the academic calendar.



Figure 8.2: Proposed Project PERT Precedence Network.

Chapter 9

Network Architecture and Security Design

9.1 Proposed Network Deployment Topology

Figure 9.1 models the network topology and trust boundaries. The frontend will communicate with the Go API over HTTP during isolated local development and HTTPS in deployment. The API and PostgreSQL will communicate over a private service network. TLS will be configured for production connections where supported by the hosting environment.

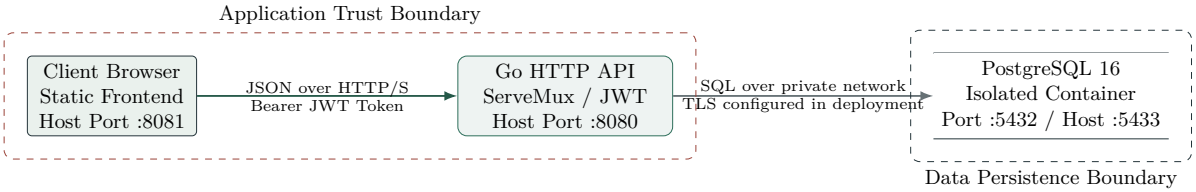


Figure 9.1: Proposed Network Deployment Topology and Trust Boundaries.

9.2 Multi-Layered Security Architecture

The planned security controls are organized across distinct architectural layers:

- **Authentication Layer:** User passwords shall be stored as adaptive bcrypt hashes, never as reversible plaintext. Successful authentication shall issue an HS256-signed JWT with a configured, documented expiration period.
- **Authorization Layer:** Protected API endpoints pass through authentication middleware that decodes and validates the JWT signature. Each protected operation shall derive its owner from the verified JWT subject; clients shall not be able to select another user by changing a path identifier.
- **Data Privacy & Minimization:** Dedicated response structures shall omit the password hash and other fields not required by the requested screen.
- **Database Security:** Local development shall use isolated container networking and environment-supplied credentials. Production deployment shall restrict database access to the API service and use managed secrets and encrypted connections where supported.
- **CORS & Session Protection:** CORS shall allow only configured frontend origins. The session design shall avoid persistent storage of bearer tokens in browser local storage. Sign-in attempts shall be rate-limited and use non-revealing error messages.

Chapter 10

Future Scope, Expected Outcomes, and Conclusion

10.1 Future Scope

Upon completion and evaluation of the primary deliverables, potential future enhancements include:

1. **Wearable IoT Telemetry Integration:** Incorporating direct Bluetooth Low Energy (BLE) synchronization with smartwatches and fitness bands.
2. **Custom Routine Builder:** Allowing advanced athletes to construct custom exercise routines and tailor repetition targets beyond pre-seeded tiers.
3. **Progressive Web App (PWA) Offline Caching:** Implementing Service Workers and IndexedDB to enable workout logging during network disconnection with background synchronization.
4. **Exportable Analytics Reports:** Adding server-side generation of downloadable PDF activity summaries and CSV data exports.

10.2 Expected Project Outcomes and Deliverables

The planned project deliverables are:

- A fully functional, responsive web frontend application implementing user registration, authentication, dashboard views, and logging workflows.
- A compiled, Go HTTP API providing robust endpoints, JWT validation, and business logic routing.
- A PostgreSQL database schema with documented integrity constraints and seeded workout and meal-plan catalogues.
- Docker and Docker Compose configuration for reproducible local database setup and optional API packaging.
- Automated tests for API authorization, database operations, and input bounds.
- Comprehensive technical documentation and a final project report detailing system evaluation.

10.3 Conclusion

The Formwell personal fitness tracking and activity monitoring system represents a well-conceived, technically viable, and academically sound project. By addressing the fragmentation of personal fitness tracking through a cohesive web architecture, compiled Go API, and normalized PostgreSQL persistence, the system demonstrates key competencies in full-stack software engineering, relational database design, and web security.

Formwell is proposed as a focused web information system for fitness-related self-logging and plan review. The project will produce an integrated implementation and an evaluation of its functional behavior, data integrity, and security controls.

Bibliography

[1] World Health Organization. *WHO Guidelines on Physical Activity and Sedentary Behaviour*. Geneva: World Health Organization; 2020. ISBN 978-92-4-001512-8. Available at: `https://www.who.int/publications/i/item/9789240015128`.

[2] Jones, M., Bradley, J., and Sakimura, N. *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force; May 2015. `https://www.rfc-editor.org/rfc/rfc7519`.

[3] GORM Authors. *Migration and Object-Relational Mapping*. GORM v2 Documentation; 2024. Available at: `https://gorm.io/docs/migration.html`.

[4] PostgreSQL Global Development Group. *Data Definition and Constraints*. PostgreSQL 16 Documentation, Chapter 5; 2024. Available at: `https://www.postgresql.org/docs/16/ddl-constraints.html`.

[5] OWASP Foundation. *Password Storage Cheat Sheet and API Security Top 10*. OWASP Cheat Sheet Series; 2023. Available at: `https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html`.

[6] Formwell project source. *Frontend README, page templates, styles, and application JavaScript*. Repository `frontend-fitness`; accessed October 2026.

[7] Formwell project source. *Backend README, Go handlers, models, and GORM database implementation*. Repository `backend-fitness`; accessed October 2026.